

impactx-ui — Roadmap do Design System

Como o bot constrói telas sem inventar UI · 5 semanas até frontend deixar de ser problema

Arquitetura em 5 camadas

#	Camada	O que é	Status hoje
5	APPS (kumon-app, futuros)	Bot escreve aqui. Só compõe, nunca inventa.	■ kumon ainda inventa UI
4	TEMPLATES	Páginas inteiras: List, Detail, Form, Dashboard	■ não existe
3	ORGANISMS + COMPONENTS	Button, Card, DataTable, HeroBanner, AssessmentCard	■ d22 base + 10 domain
2	LAYOUT PRIMITIVES	<Grid>, <Stack>, <PageContainer>, <Cluster>	■ não existe
1	TOKENS (cor, radius, spacing, tipografia)	OSs definidos por tema (alfabeto / kumon / impactx)	■ funciona (refatorar p/ 3 níveis depois)

Lê de baixo pra cima: cada camada consome só a de baixo. Bot nos apps (topo) compõe templates + organisms; nunca toca em tokens crus nem inventa layout.

Mental model: quem decide o quê

Decisão	Quem decide	Onde vive
Cor, radius, spacing, tipografia	Token	tokens/themes/*.css
Como Button se comporta	Componente	ds-education/button.tsx
Grid responsivo, stack mobile	Layout primitive	<Grid>, <Stack>
Tabela com paginação e filtros	Organism composto	<DataTableWithPagination>
Estrutura de página de lista	Template	<ListPageTemplate>
Qual aluno mostrar, qual ação rodar	App (bot)	kumon-app/app/alunos

Antes vs Depois: como o bot escreve uma tela

■ HOJE — bot inventa CSS	■ DEPOIS — bot compõe DS
<pre><div className="max-w-7xl mx-auto px-4 py-6"> <div className="flex justify-between mb-4"> <h1 className="text-2xl font-bold">Alunos</h1> <button className="bg-blue-500 px-4 py-2 rounded-lg text-white">Novo aluno</button> </div> <div className="grid grid-cols-3 gap-4"> /* tabela manual, sem paginação */ </div> </div></pre>	<pre><ListPageTemplate title="Alunos" primaryAction={{ label: "Novo aluno", onClick: handleNew }} table={{ <DataTableWithPagination columns={columns} data={alunos} /> }} /></pre>
~12 linhas · CSS cru · grid quebra em mobile · tema kumon não aplicado · sem dark mode · paginação não existe · acessibilidade zero	~10 linhas · zero CSS · responsivo nativo · tema kumon automático · dark mode automático · paginação embutida · a11y por padrão

Mesma tela, mesmo resultado visual — mas a 2ª versão é impossível de quebrar responsividade, tema ou consistência. Bot vira compositor, não inventor.

Ordem de build — 5 semanas

Sem.	O que entrega	Por que primeiro	Impacto
1	Layout primitives: <Grid> <Stack> <PageContainer> <Cluster> + <DataTableWithPagination>	Sem isso, bot inventa grid e quebra responsividade. Os audits já apontaram (responsive-rootcause).	Desbloqueia tudo
2	Templates de página: ListPage, DetailPage, FormPage, DashboardPage	4 templates cobrem 90% das telas. Bot escreve 10 linhas, recebe página inteira responsiva.	Telas viram plug-and-play
3	Tokens em 3 níveis (Style Dictionary): primitive → semantic → component	Agora componentes existem pra consumir. Mudar 1 token propaga em N apps automaticamente.	“Borda mais redonda” = 1 linha
4	Versionamento (Changesets) + Visual regression (Playwright no apps/web showcase)	Protege contra regressão visual. Bot atualiza DS com diff automático antes de propagar.	Atualização sem medo
5	Kumon migra pro DS + ESLint rule (bloqueia Tailwind cru fora dos primitives)	Bot literalmente não consegue escrever CSS inconsistente — lint falha antes de commitar.	Bot incapaz de errar

Decisão chave: primitives antes de tokens 3-níveis. Tokens hoje **já funcionam** (CSS vars + 3 temas). Refatorar tokens sem ter primitives pra consumir é pintar parede que vai cair. Primitives são o gap real do bot.

Sobre organisms: sim — os 10 components-education/ (hero-banner, assessment-card, big-card) são organisms de domínio. Vivem no DS, não nos apps. Bot importa e compõe. Mental model correto.